```
#Module 3 Lab 1
#Introduction to Seaborn

#Installation
#pip install seaborn
```

Seaborn2

Figure

Figure

It refers to the whole figure that you see. It is possible to have multiple sub-plots (Axes) in the same figure.

Axes

An Axes refers to the actual plot in the figure. A figure can have multiple Axes but a given Axes can be part of only one figure.

Axis

An Axis refers to an actual axis (x-axis/y-axis) in a specific plot.

Four sub-plots (Axes) in a single figure.

Figure2

Seaborn

Seaborn - can create complicated plot types from Pandas data with relatively simple commands

Plotting in seaborn is either : Axes-level functions OR Figure-level function

PLOT CATEGORIES IN SEABORN

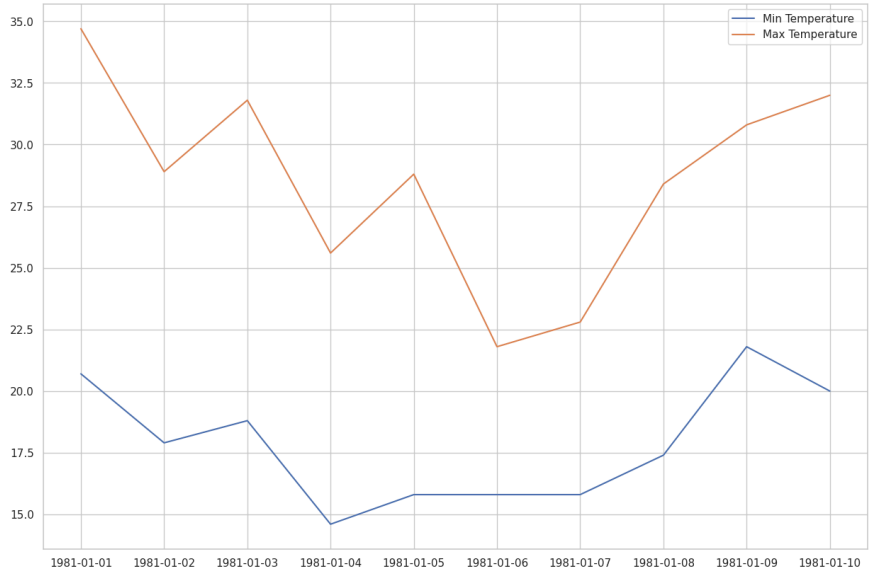
I. Relational plots : This plot is used to understand the relation between two variables.
II. Categorical plots: This plot deals with categorical variables and how they can be visualized.
III. Distribution plots: This plot is used for examining univariate and bivariate distributions
IV. Matrix plots: A matrix plot is an array of scatterplots.
V. Regression plots: The regression plots in seaborn are primarily intended to add a visual guide that helps to emphasize patterns in a dataset during exploratory data analyses.

Seaborn

```
#Import necessary Packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.pyplot import figure

import seaborn as sns

%matplotlib inline


#Simple Plotting with Seaborn

#Data
dates = ['1981-01-01', '1981-01-02', '1981-01-03', '1981-01-04', '1981-01-05',
        '1981-01-06', '1981-01-07', '1981-01-08', '1981-01-09', '1981-01-10' ]

min_temperature = [20.7, 17.9, 18.8, 14.6, 15.8, 15.8, 15.8, 17.4, 21.8, 20.0]
max_temperature = [34.7, 28.9, 31.8, 25.6, 28.8, 21.8, 22.8, 28.4, 30.8, 32.0]

#Plotting
fig,axes = plt.subplots(nrows=1, ncols=1, figsize=(15,10))
```

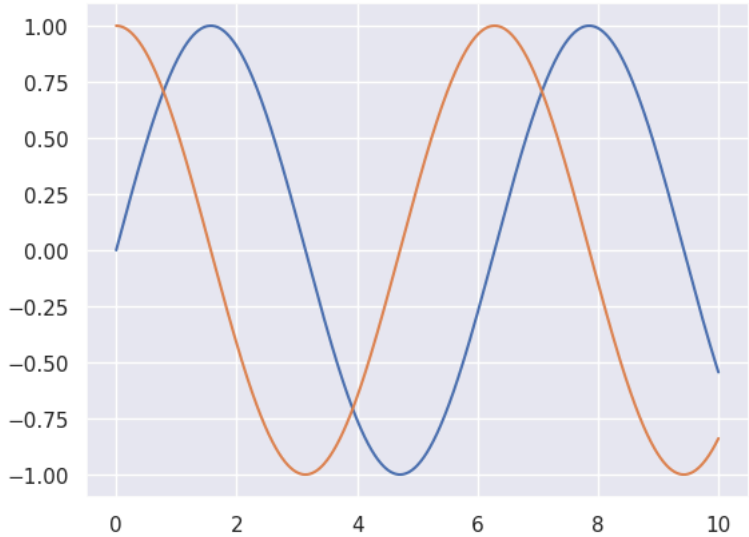
```
fig,axes = plt.subplots(nrows=1, ncols=1, figsize=(15,10))
axes.plot(dates, min_temperature, label='Min Temperature')
axes.plot(dates, max_temperature, label = 'Max Temperature')
axes.legend()
```

```
<matplotlib.legend.Legend at 0x7fd41a78c880>
```




```
#seaborn style as the default matplotlib style
sns.set()
```

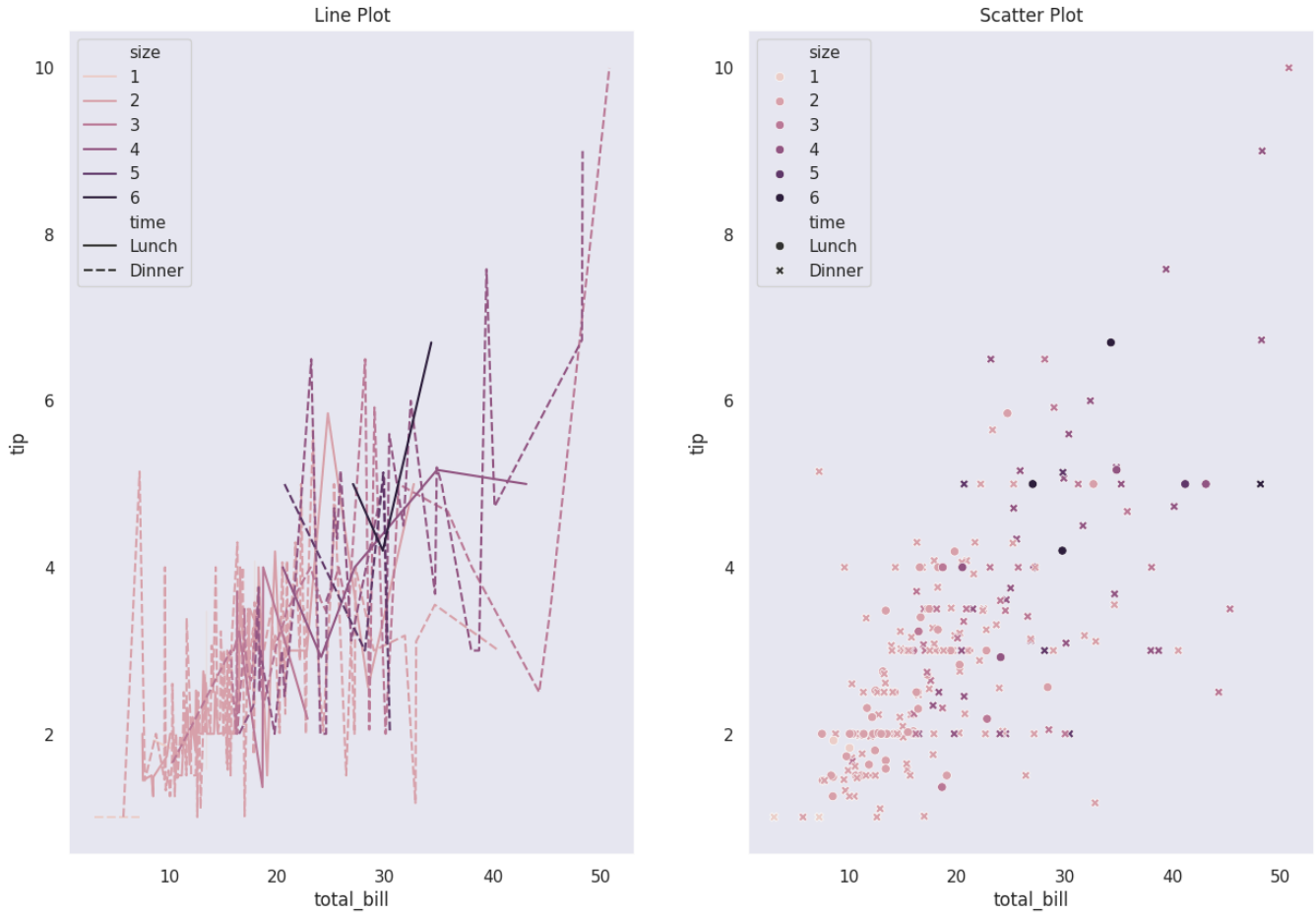
```
#Simple sine plot
x = np.linspace(0, 10, 1000)
plt.plot(x, np.sin(x), x, np.cos(x));
```




```
# I. Relational Plots
```

```
# Line plot : The line plot is one of the most basic plot in seaborn library.
#This plot is mainly used to visualize the data in form of some time series, i.e. in continuous manner.
sns.set(style="dark")
fig, ax = plt.subplots(ncols=2, nrows=1, figsize=(15,10))

#Loading Data with Seaborn
df = sns.load_dataset("tips")

print(df.head())

#lineplot
sns.lineplot(x="total_bill", y="tip", hue="size", style="time", data=df,ax=ax[0]).set_title("Line Plot")


#scatterplot
Sct_plt=sns.scatterplot(x="total_bill", y="tip", hue="size", style="time", data=df,ax=ax[1]).set_title("Scatter Plot")

#Saving Plot
Sct_plt.figure.savefig('Scatter_plot1.png')
print('Plot Saved')
```

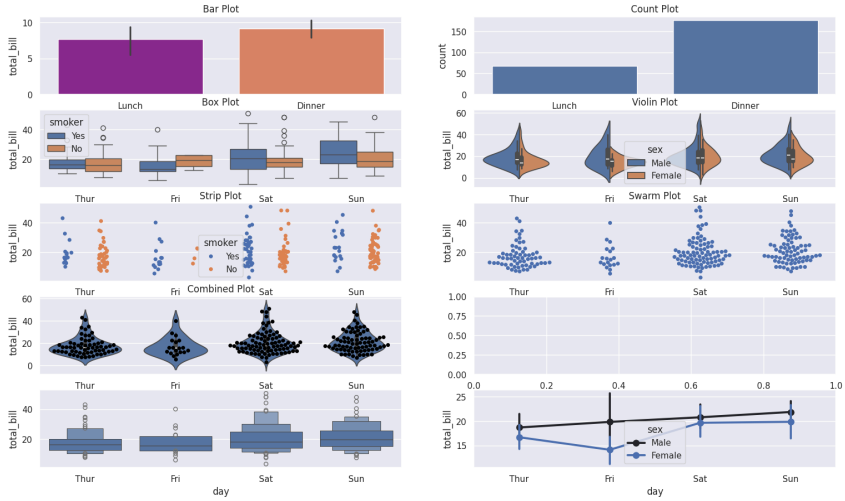
```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
Plot Saved
```

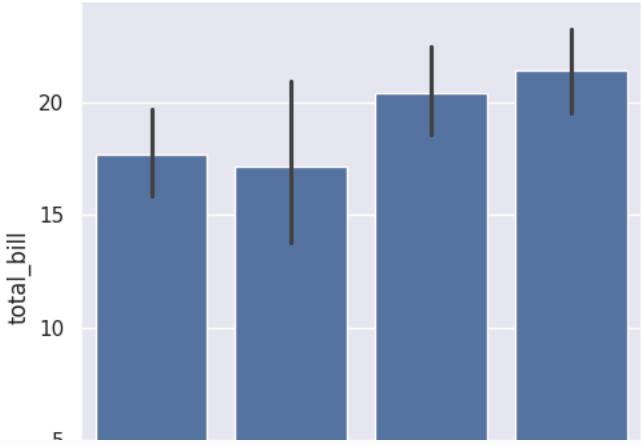



```
#II. Categorical Plots
#Plots are basically used for visualizing the relationship between variables.
#Variables can be either be completely numerical or a category like a group, class or division.

sns.set_style('darkgrid')
fig, ax =plt.subplots(nrows=5,ncols=2)
```

```
fig.set_size_inches(18.5, 10.5)

#Data
# 'tips’ dataset contains information about people who probably had food at a restaurant
#  whether or not they left a tip for the waiters, their gender, whether they smoke and so on.
df = sns.load_dataset('tips')

#barplot -  basically used to aggregate the categorical data according to some methods and by default its the mean
sns.barplot(x ='time', y ='total_bill', data = df,palette ='plasma', estimator = np.std,ax=ax[0,0]).set_title('Bar Plot')


#countplot -Counts the categories and returns a count of their occurrences
sns.countplot(x ='time', data = df,ax=ax[0,1]).set_title('Count Plot')

#boxplot - known as the box and whisker plot.
#It shows the distribution of the quantitative data that represents the comparisons between variables
sns.boxplot(x ='day', y ='total_bill', data = df, hue ='smoker',ax=ax[1,0]).set_title('Box Plot')

# Similar to the boxplot except that it provides a higher, more advanced visualization
# Uses the kernel density estimation to give a better description about the data distribution.
sns.violinplot(x ='day', y ='total_bill', data = df, hue ='sex', split = True,ax=ax[1,1]).set_title('Violin Plot')

#Stripplot -  scatter plot based on the category
sns.stripplot(x ='day', y ='total_bill', data = df, jitter = True, hue ='smoker', dodge = True,ax=ax[2,0]).set_title('Strip Plot')

#Swarmplot-similar to stripplot except the fact that the points are adjusted so that they do not overlap.
sns.swarmplot(x ='day', y ='total_bill', data = df,ax=ax[2,1]).set_title('Swarm Plot')

#Combining the idea of a violin plot and a stripplot to form this plot
sns.violinplot(x ='day', y ='total_bill', data = df,ax=ax[3,0])
sns.swarmplot(x ='day', y ='total_bill', data = df, color ='black',ax=ax[3,0]).set_title('Combined Plot')

# Density Plot
#sns.kdeplot(df['tip'], df['total_bill'],ax=[3,1])

#boxenplot
sns.boxenplot(x="day", y="total_bill",color="b", scale="linear", data=df,ax=ax[4,0])

#Ridgeplot
sns.pointplot(x="day", y="total_bill",color="b", hue="sex", data=df,ax=ax[4,1])

#catplot
#General plot -  provides a parameter called 'kind' to choose the kind of plot ,better that writing the plots separately.
#The kind parameter can be bar, violin, swarm etc.
sns.catplot(x ='day', y ='total_bill', data = df, kind ='bar')
```

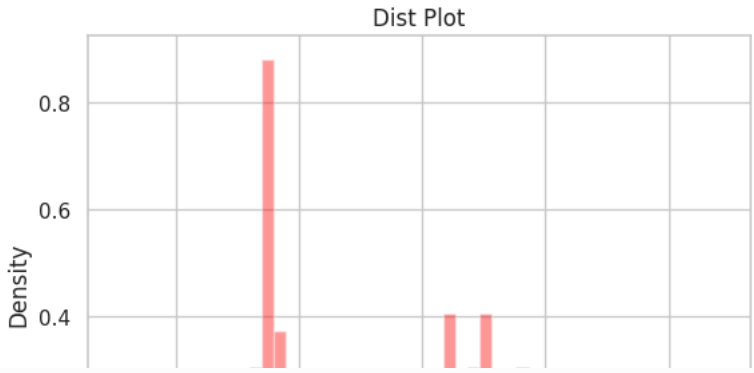
```
<ipython-input-34-3c98d967bba2>:15: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.1

  sns.barplot(x ='time', y ='total_bill', data = df,palette ='plasma', estimator =
<ipython-input-34-3c98d967bba2>:43: FutureWarning:

The `scale` parameter has been renamed to `width_method` and will be removed in v0
  sns.boxenplot(x="day", y="total_bill",color="b", scale="linear", data=df,ax=ax[4,
<ipython-input-34-3c98d967bba2>:46: FutureWarning:

Setting a gradient palette using color= is deprecated and will be removed in v0.14

  sns.pointplot(x="day", y="total_bill",color="b", hue="sex", data=df,ax=ax[4,1])
<seaborn.axisgrid.FacetGrid at 0x7fd416b07610>
```





III. **Distribution plots** in seaborn is used for examining univariate and bivariate distributions. 4 main types of distribution plots :

```
joinplot
distplot
pairplot
rugplot
```

```
sns.set_style('whitegrid')


#Data - 'iris'
df = sns.load_dataset('iris')
print(df.head())

#Displot- used for univariant set of observations and visualizes it through a histogram
#i.e. only one observation and hence we choose one particular column of the dataset.
#KDE is a way to estimate the probability density function (PDF) of the random variable that “underlies” the sample.
#KDE is a means of data smoothing.
#bins is used to set the number of bins you want in your plot and it actually depends on your dataset.
#color is used to specify the color of the plot
sns.distplot(df['petal_length'], kde = True, color ='red', bins = 30).set_title('Dist Plot')

#Joinplot/jointgrid- draw a plot of two variables with bivariate and univariate graphs. It basically combines two different plots.
#Plot a bi-variate distribution along with marginal distributions in the same plot
#Joint Distribution of two variables can be visualised using scatter plot/regplot or kdeplot.
#Marginal Distribution of variables can be visualised by histograms and/or kde plot
#KDE shows the density where the points match up the most
#The Axes-level function to use for joint distribution must be passed to JointGrid.plot_joint().
#The Axes-level function to use for marginal distribution must be passed to JointGrid.plot_marginals()

jointgrid = sns.JointGrid(x='petal_length', y='petal_width', data=df)
jointgrid.plot_joint(sns.scatterplot)
jointgrid.plot_marginals(sns.distplot)

#jointplot() to plot bi-variate distribution along with marginal distributions.
#It uses JointGrid() and JointGrid.plot_joint() in the background.
g=sns.jointplot(x = 'petal_length',y = 'petal_width',data = df,kind = 'hex')
g.fig.suptitle('Joint Plot')

#Pairplot-  pairwise relation across the entire dataframe
#hue sets up the categorical separation between the entries in the dataset.
#palette is used for designing the plots.
g=sns.pairplot(df, hue ="species", palette ='coolwarm')
g.fig.suptitle("Pair Plot 1")
g.add_legend()


#PairGrid() - creates Axes for each pair of variables
#PairGrid.map() - draws the plot on each Axes using data corresponding to that pair of variables
pairgrid = sns.PairGrid(data=df)
pairgrid = pairgrid.map_offdiag(sns.scatterplot)
pairgrid = pairgrid.map_diag(plt.hist)

#Different kind of plots on Upper Triangular Axes, Diagonal Axes and Lower Triangular Axes.
pairgrid = sns.PairGrid(data=df)
pairgrid = pairgrid.map_upper(sns.scatterplot)
pairgrid = pairgrid.map_diag(plt.hist)
pairgrid = pairgrid.map_lower(sns.kdeplot)

#Avoid Redundancy
g = sns.PairGrid(df, diag_sharey=False, corner=True)
g.map_lower(sns.scatterplot)
g.map_diag(sns.kdeplot)
```

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
<ipython-input-9-4514184d181a>:14: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(df['petal_length'], kde = True, color ='red', bins = 30).set_title(
/usr/local/lib/python3.10/dist-packages/seaborn/axisgrid.py:1886: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  func(self.x, **orient_kw_x, **kwargs)
/usr/local/lib/python3.10/dist-packages/seaborn/axisgrid.py:1892: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  func(self.y, **orient_kw_y, **kwargs)
<seaborn.axisgrid.PairGrid at 0x7fd41b33bd60>
```